File Handling in Java

27 June 2024 09:51

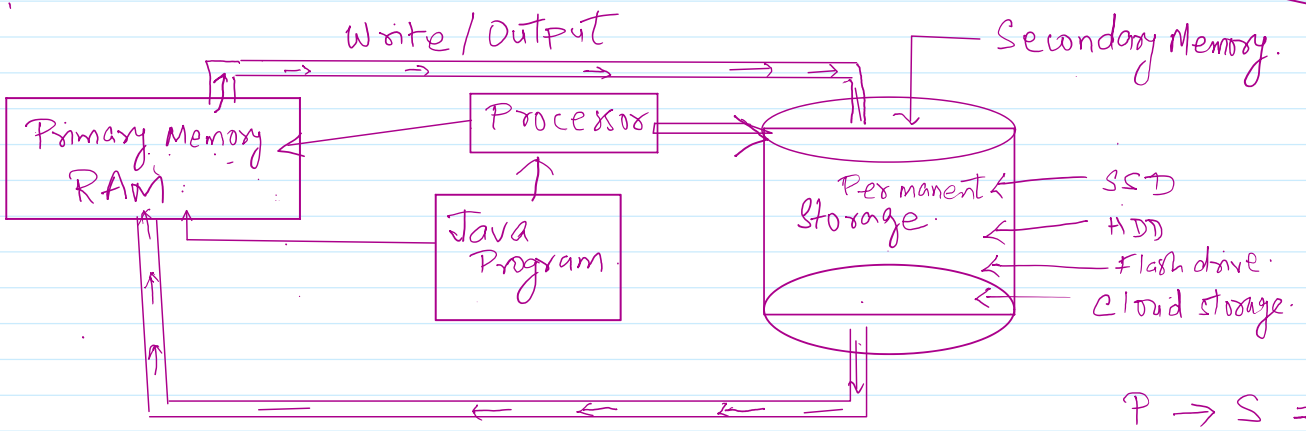
File handling in Java is a crucial aspect of the language enabling programs to read from and write to files. Java provides a rich set of classes and methods to handle file operations primarily located in the **'java.io'** and **'java.nio'** packages.

Reading data on the primary memory.

is used for writing data on the secondary memory

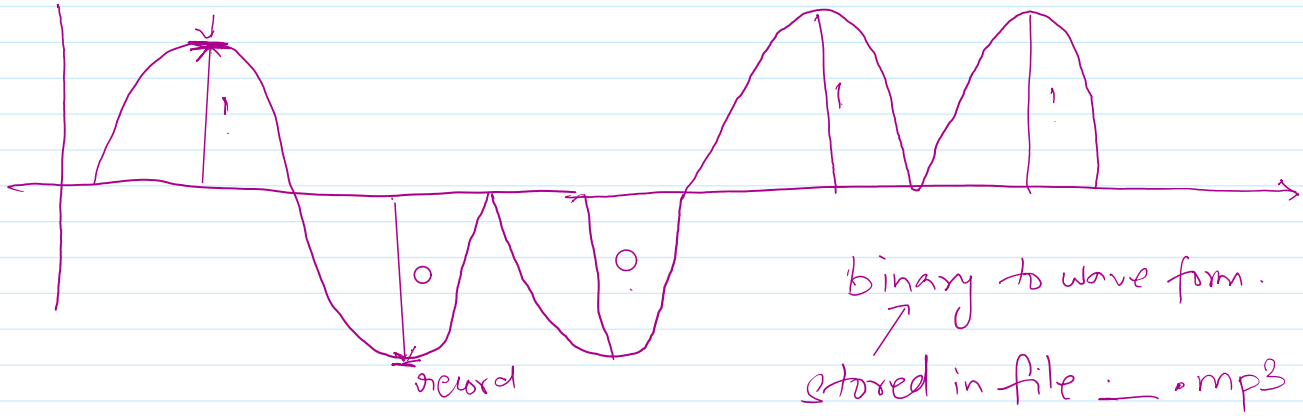
Basic Concept

- **File:** An abstract representation of file and directory path names.
- **Stream:** A sequence of data in Java. in Java, there are two main types of stream.
  - **Byte Streams:** Handle data in binary format. Example, (**InputStream** and **OutputStream**).
  - **Character streams:** Handle data in character format examples (**Reader** and **Writer**).



P → S = write / output

S → P = read / input.



binary to wave form.

stored in file — .mp3
— .ogg
— .wav
.class
.exe.

Character files have extension .txt
.java
.c
.cpp
.js
.html
.css
⋮

Commonly used classes in java.io

1. **File**
2. **FileInputStream**
3. **FileOutputStream**
4. **FileReader**
5. **FileWriter**
6. **BufferedReader**
7. **BufferedWriter**
8. **PrintWriter**

'File' Class

The file class is used to represent file and directory path names in an abstract manner.
Read the File class in detail, use the link.
https://docs.oracle.com/javase%2F8%2Fdocs%2Fapi%2F%2F/java/io/File.html

```
package FileHandling;
import java.io.*;
import java.util.*;

/**
 * Write a description of class CreateFileUsinfFileClass here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class CreateFileUsingFileClass
{
   public static void main(String args[])  {
      Scanner sc = new Scanner(System.in);
      String fileName;
      System.out.print("\fEnter file name: ");
      fileName = sc.next();
      try{
         File file = new File("./Data/" + fileName);
         if(file.createNewFile()){
            System.out.print("\nFile created: " +
               file.getAbsolutePath());
         }
         else{
            System.out.print("\nFile already exists.");
         }
      }
      catch(IOException e){
         System.err.print("\nAn Error Occurred!");
         e.printStackTrace();
      }
   }
}
```

The 'file' object is created with the specified file name.

The 'createNewFile()' method attempts to create the file, if successful, it returns 'true' otherwise false.

The 'createNewFile()' method creates the File one the specified valid path, if the file doesn't exists.

Writing to a File using FileWriter and BufferedWriter class.

'FileWriter' class

The **'FileWriter'** class is used to write characters to a file. It is a simple class designed for writing streams of characters. When you write a file using 'FileWriter', you can specify whether to overwrite a file or append

to it.

```
package FileHandling;
import java.util.*;
import java.io.*;

/**
 * Write a description of class WritingToFile here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class WritingToFile
{
   public static void main(String args[]){
      //approach-1 using File object
      String path, fileName;
      Scanner sc = new Scanner(System.in);
      path = "./Data/";
      System.out.print("\fEnter filename: ");
      fileName = sc.nextLine();
      File file;
      FileWriter fw = null;
      try{
         file = new File(path + fileName);
         //fw = new FileWriter(file); //not append mode data is overwritten
         fw = new FileWriter(file, true); //append mode data is append at the end of existing data.
         while(true){
            String line = sc.nextLine()+"\n";
            fw.write(line);
            System.out.print("\nFor more lines press 'Y/n': ");
            line = sc.nextLine();
            if(line.equalsIgnoreCase("n")){
               break;
            }
         }
      }
      catch(IOException e){
         System.err.print("\nFile not created!");
      }
      finally{
         try
         {
            fw.close();
         }
         catch (IOException ioe)
         {
            ioe.printStackTrace();
         }
         sc.close();

      }
   }
}
```

method writes the specified string to the file.

Second Approach using FileWriter(String fileName, boolean mode), if mode = true then append mode is

on.

```
package FileHandling;

import java.util.*;
import java.io.*;

/**
 * Write a description of class WritingToFile here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class WritingToFile2
{
   public static void main(String args[]){
      //approach-2
      String path, fileName;
      Scanner sc = new Scanner(System.in);
      path = "./Data/";
      System.out.print("\fEnter filename: ");
      fileName = sc.nextLine();
      FileWriter fw = null;
      try{
         //fw = new FileWriter(file); //not append mode
         //Constructor FileWriter(String fileName, mode);
         fw = new FileWriter(path+fileName, true); //append mode
         while(true){
            String line = sc.nextLine()+"\n";
            //fw.write(line);
            fw.append(line);
            System.out.print("\nFor more lines press 'Y/n': ");
            line = sc.nextLine();
            if(line.equalsIgnoreCase("n")){
               break;
            }
         }
      }
      catch(IOException e){
         System.err.print("\nFile not created!");
      }
      finally{
         try
         {
            fw.close();
         }
         catch (IOException ioe)
         {
            ioe.printStackTrace();
         }
         sc.close();

      }
   }
}
```